```cpp
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int f[N][N]; // 差分矩阵

// 差分数组的性质
void insert(int x1, int y1, int x2, int y2, int k)
{
    f[x1][y1] += k; f[x1][y2 + 1] -= k; f[x2 + 1][y1] -= k; f[x2 + 1][y2 + 1] += k;
}

int main()
{
    cin >> n >> m;

    // 构建差分数组
    while(m--)
    {
        int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        insert(x1, y1, x2, y2, 1);
    }

    // 利用前缀和还原修改之后的数组
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + f[i][j];
            cout << f[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

6. 双指针

> 双指针算法有时候也叫**尺取法**或者**滑动窗口**，是**一种优化暴力枚举策略的手段**：

- 当我们发现在两层 for 循环的暴力枚举过程中，**两个指针是可以不回退的**，此时我们就可以利用两个指针不回退的性质来优化时间复杂度。
- 因为双指针算法中，两个指针是朝着同一个方向移动的，因此也叫做**同向双指针**。

注意：希望大家在学习该算法的时候，不要只是去记忆模板，一定要学会**如何从暴力解法优化成双指针算法**。不然往后遇到类似题目，你可能压根都想不到用双指针去解决。

6.1 唯一的雪花

题目来源： 洛谷

题目链接： **唯一的雪花 Unique Snowflakes**

难度系数： ★★

【题目描述】

企业家 Emily 有一个很酷的主意：把雪花包起来卖。她发明了一台机器，这台机器可以捕捉飘落的雪花，并把它们一片一片打包进一个包裹里。一旦这个包裹满了，它就会被封上送去发售。

Emily 的公司的口号是“把独特打包起来”，为了实现这一诺言，一个包裹里不能有两片一样的雪花。不幸的是，这并不容易做到，因为实际上通过机器的雪花中有很多是相同的。Emily 想知道这样一个不包含两片一样的雪花的包裹最大能有多大，她可以在任何时候启动机器，但是一旦机器启动了，直到包裹被封上为止，所有通过机器的雪花都必须被打包进这个包裹里，当然，包裹可以在任何时候被封上。

【输入描述】

第一行是测试数据组数 T，对于每一组数据，第一行是通过机器的雪花总数 $n(n \leq 10^6)$，下面 n 行每行一个在 $[1, 10^9]$ 内的整数，标记了这片雪花，当两片雪花标记相同时，这两片雪花是一样的。

【输出描述】

对于每一组数据，输出最大包裹的大小。

【示例一】

输入：

```
1
5
1
2
3
2
```

```
1
```

输出：

```
3
```

【解法】

当我们「暴力枚举」的过程中，固定一个起点位置 $left$ ，然后 $right$ 之后向后遍历时。当 $right$ 第一次扫描到一个位置，使 $[left, right]$ 这个区间「出现重复字符」，此时我们会发现：

- $right$ 无需再向后遍历，因为继续向后走也是「不合法」的；
- $left$ 向后移动一格之后， $right$ 指针也不用回退，因为我们已经维护出来 $[left, right]$ 区间的信息，并且以 $left + 1$ 为起点的最优解一定不会比 $left$ 为起点的好。

当我们发现暴力枚举的「两个指针不回退」时，就可以用「滑动窗口」优化：

- 进窗口： $right$ 位置元素记录到统计次数的哈希表中；
- 判断：当哈希表中 $right$ 位置的值出现超过 1 次之后，窗口内子串不合法；
- 出窗口：让 $left$ 所指位置的元素在哈希表中的次数减一；
- 更新结果：判断结束之后，窗口合法，此时更新窗口的大小。

【参考代码】

代码块

```cpp
#include <iostream>
#include <unordered_map>

using namespace std;

const int N = 1e6 + 10;

int n;
int a[N];

int main()
{
    int T; cin >> T;

    while(T--)
    {
        cin >> n;
        for(int i = 1; i <= n; i++) cin >> a[i];

```

```
        // 初始化
        int left = 1, right = 1, ret = 0;
        unordered_map<int, int> mp; // 维护窗口内所有元素出现的次数
        while(right <= n)
        {
            // 进窗口
            mp[a[right]]++;
            // 判断
            while(mp[a[right]] > 1)
            {
                // 出窗口
                mp[a[left]]--;
                left++;
            }
            // 窗口合法，更新结果
            ret = max(ret, right - left + 1);
            right++;
        }

        cout << ret << endl;
    }

    return 0;
}
```

6.2 逛画展

> **题目来源：** 洛谷
>
> **题目链接：** P1638 逛画展
>
> **难度系数：** ★★

【题目描述】

> 博览馆正在展出由世上最佳的 m 位画家所画的图画。
>
> 游客在购买门票时必须说明两个数字， a 和 b ，代表他要看展览中的第 a 幅至第 b 幅画（包含 a, b）之间的所有图画，而门票的价钱就是一张图画一元。
>
> Sept 希望入场后可以看到所有名师的图画。当然，他想最小化购买门票的价格。
>
> 请求出他购买门票时应选择的 a, b，数据保证一定有解。
>
> 若存在多组解，输出 a 最小的那组。

【输入描述】

> 第一行两个整数 n, m，分别表示博览馆内的图画总数及这些图画是由多少位名师的画所绘画的。

> 第二行包含 n 个整数 a_i，代表画第 i 幅画的名师的编号。

- 对于 100% 的数据，有 $1 \le n \le 10^6, 1 \le a_i \le m \le 2 \times 10^3$。

【输出描述】

> 一行两个整数 a, b。

【示例一】

> 输入：
>
> ```
> 12 5
> 2 5 3 1 3 2 4 1 1 5 4 3
> ```
>
> 输出：
>
> ```
> 2 7
> ```

【解法】

当我们「暴力枚举」的过程中，固定一个起点位置 $left$，然后 $right$ 之后向后遍历时。当 $right$ 第一次扫描到一个位置，使 $[left, right]$ 这个区间已经「包含了所有的数时」，此时我们会发现：

- $right$ 无需再向后遍历，因为继续向后走肯定不是「最优解」；
- $left$ 向后移动一格之后， $right$ 指针也不用回退，因为我们已经维护出来 $[left, right]$ 区间的信息，能够快速得出 $[left + 1, right]$ 区间「是否合法」，而且最「优右端点」铁定不在 $right$ 左边。

当我们发现「暴力枚举」的两个指针「不回退」时，就可以用「滑动窗口」优化：

- 进窗口： $right$ 位置元素记录到统计次数的哈希表中，如果次数是 0 变 1 ，说明窗口内多了「一种」字符，记录一下；
- 判断：当窗口内字符种类等于 m 时，窗口合法， $right$ 停止右移，接下来需要出窗口；
- 出窗口：让 $left$ 所指位置的元素在哈希表中的次数减一，如果次数是 1 变 0 ，说明窗口内少了「一种」字符，记录一下；
- 更新结果：在判断成立时，窗口合法，此时更新窗口的大小。

【参考代码】

代码块

```
#include <iostream>

using namespace std;
```

```cpp

const int N = 1e6 + 10, M = 2e3 + 10;

int n, m;
int a[N];

int kind; // 窗口内有效元素的个数
int mp[N]; // 统计窗口内每个元素出现的次数

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];

    int left = 1, right = 1;
    int ret = n, begin = 1;

    while(right <= n)
    {
        // 进窗口
        if(mp[a[right]]++ == 0) kind++;
        // 判断
        while(kind == m)
        {
            // 更新结果
            int len = right - left + 1;
            if(len < ret)
            {
                ret = len;
                begin = left;
            }
            // 出窗口
            if(mp[a[left]]-- == 1) kind--;
            left++;
        }
        right++;
    }

    cout << begin << " " << begin + ret - 1 << endl;

    return 0;
}
```

6.3 字符串

> **题目来源：** 牛客网
>
> **题目链接：** 字符串
>
> **难度系数：** ★★

【题目描述】

> 小 N 现在有一个字符串 S。他把这这个字符串的所有子串都挑了出来。一个 S 的子串 T 是合法的，当且仅当 T 中包含了所有的小写字母。小 N 希望知道所有的合法的 S 的子串中，长度最短是多少。

【输入描述】

> 一行一个字符串 S。只包含小写字母。S 的长度不超过 10^6 .

【输出描述】

> 一行一个数字，代表最短长度。数据保证存在一个合法的 S 的子串。

【示例一】

> 输入：
>
> ykjygvedtysvyymzfizzwkjamefxjnrnphqwnfhrnbhwjhqcgqnplodeestu
>
> 输出：
>
> 49

【解法】

当我们「暴力枚举」的过程中，固定一个起点位置 $left$ ，然后 $right$ 之后向后遍历时。当 $right$ 第一次扫描到一个位置，使 $[left, right]$ 这个区间已经「包含了所有的小写字母时」，此时我们会发现：

- $right$ 无需再向后遍历，因为继续向后走肯定不是「最优解」；
- $left$ 向后移动一格之后， $right$ 指针也不用回退，因为我们已经维护出来 $[left, right]$ 区间的信息，能够快速得出 $[left + 1, right]$ 区间「是否合法」，而且最「优右端点」铁定不在 $right$ 左边。

当我们发现「暴力枚举」的两个指针「不回退」时，就可以用「滑动窗口」优化：

- 进窗口： $right$ 位置元素记录到统计次数的哈希表中，如果次数是 0 变 1 ，说明窗口内多了「一种」字符，记录一下；
- 判断：当窗口内字符种类等于 m 时，窗口合法， $right$ 停止右移，接下来需要出窗口；
- 出窗口：让 $left$ 所指位置的元素在哈希表中的次数减一，如果次数是 1 变 0 ，说明窗口内少了「一种」字符，记录一下；
- 更新结果：在判断成立时，窗口合法，此时更新窗口的大小。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

string s;
int mp[26]; // 统计每个小写字符出现的次数
int kind; // 窗口内小写字符的种类

int main()
{
    cin >> s;
    int n = s.size();
    int ret = n;

    // 初始化
    for(int left = 0, right = 0; right < n; right++)
    {
        // 进窗口
        if(mp[s[right] - 'a']++ == 0) kind++;
        // 判断
        while(kind == 26)
        {
            // 更新结果
            ret = min(ret, right - left + 1);
            // 出窗口
            if(mp[s[left] - 'a']-- == 1) kind--;
            left++;
        }
    }

    cout << ret << endl;

    return 0;
}
```

6.4 丢手绢

> **题目来源：** 牛客网
>
> **题目链接：** 丢手绢

> **难度系数：** ★★★

【题目描述】

> “丢~丢~丢手绢，轻轻地放在小朋友的后面，大家不要告诉她，快点快点抓住她，快点快点抓住她。”
>
> 牛客幼儿园的小朋友们围成了一个圆圈准备玩丢手绢的游戏，但是小朋友们太小了，不能围成一个均匀的圆圈，即每个小朋友的间隔可能会不一致。为了大家能够愉快的玩耍，我们需要知道离得最远的两个小朋友离得有多远（如果太远的话牛老师就要来帮忙调整队形啦！）。
>
> 因为是玩丢手绢，所以小朋友只能沿着圆圈外围跑，所以我们定义两个小朋友的距离为沿着圆圈顺时针走或者逆时针走的最近距离。

【输入描述】

> 第一行一个整数 N，表示有 N 个小朋友玩丢手绢的游戏。
>
> 接下来的第 2 到第 n 行，第 i 行有一个整数，表示第 $i-1$ 个小朋友顺时针到第 i 个小朋友的距离。
>
> 最后一行是第 N 个小朋友顺时针到第一个小朋友的距离。

【输出描述】

> 输出一个整数，为离得最远的两个小朋友的距离。

【示例一】

> 输入：
>
> ```
> 3
> 1
> 2
> 3
> ```
>
> 输出：
>
> ```
> 3
> ```

【解法】

处理环的一个技巧，把环分成两部分分析：



当我们「暴力枚举」的过程中，固定一个起点位置 $left$，然后 $right$ 之后向后遍历时，记 k 为 $[left, right]$ 之间的距离。当 $right$ 第一次扫描到 $k \times 2 \geq sum$ 时，此时我们会发现：

- $right$ 无需再向后遍历，因为继续向后走的结果一定不是最优的；
- $left$ 向后移动一格之后， $right$ 指针也不用回退，因为我们已经维护出来 $[left, right]$ 区间的信息， $right$ 回退也不是最优解。

当我们发现暴力枚举的「两个指针不回退」时，就可以用「滑动窗口」优化：

- 进窗口： $right$ 位置与前一个位置的距离累加到 k 中；
- 判断： $k \times 2 \geq sum$ 时，此时 $right$ 指针不用前进，应该让 $left$ 所指的元素出窗口
- 出窗口：让 $left$ 所指位置与前一个位置的距离累减到 k 中；
- 更新结果：需要在两个地方更新：
  a. 判断结束之后，此时 $[left, right]$ 之间可能是最优解，用 k 更新结果；
  b. 判断成立的时候，此时 $[right, left]$ 之间可能是最优解，用 $sum - k$ 更新结果。

【参考代码】

代码块

```
#include <iostream>

using namespace std;
```

```cpp

typedef long long LL;

const int N = 1e5 + 10;

int n;
LL a[N];

int main()
{
    cin >> n;
    LL sum = 0;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i];
        sum += a[i];
    }

    int left = 1, right = 1;
    LL k = 0;
    LL ret = 0;

    while(right <= n)
    {
        k += a[right];
        while(2 * k >= sum)
        {
            // 用 sum - k 来更新结果
            ret = max(ret, sum - k);
            k -= a[left++];
        }

        // 用 k 来更新结果
        ret = max(ret, k);
        right++;
    }

    cout << ret << endl;

    return 0;
}
```

7. 二分算法